

HAAIS-DSL: DSL to develop Home Automation and Ambient Intelligence Systems*

Pedro J. Clemente
Univertiy of Extremadura
QuercusSE Group
pjcleme@unex.es

José M. Conejero
University of Extremadura
<http://quercusseg.unex.es>
chemacm@unex.es

Juan Hernández
University of Extremadura
QuercusSE Group
juanher@unex.es

Lara Sánchez
University of Extremadura
Av. Universidad s/n
Cáceres, Spain
larasg@unex.es

ABSTRACT

Domain Specific Language (DSL) is an emergent software engineering discipline that allows software architects to model systems based on the elements of a specific domain. Home Automation (HA) and Ambient Intelligence (AmI) are examples of specific domains and they are considered the key elements in the future of home development. However, software for these domains is usually hand coded based on embedded devices and specific implementation technologies and frameworks. In this paper we present a Model Driven Development (MDD) approach to develop software systems for HA and AmI. A Domain Specific Language has been designed to model the architecture of these kinds of systems. Then, taking as input the architecture models, a set of model transformations allows code and configuration generation for a specific device platform like KNX/EIB (European Installation Bus).

Keywords

DSL, Model Driven Development, Home Automation, Ambient Intelligence, Embedded devices, Code Generation.

1. INTRODUCTION

In the last years, future homes and buildings have been idealized in the film industry. These films have shown how people and intelligent buildings interact with each other. Recently, the number of devices introduced at homes has considerably increased making the introduction of these functionalities in any building feasible. These devices must be properly coordinated in order to achieve a high degree of

comfort, security and energy savings. In that sense, the new concept of Ambient Intelligence (AmI in the rest of paper [5, 11]) refers to the introduction of several technologies into buildings, extending the traditional idea of Home Automation (HA in the rest of paper). In particular, AmI improves the services that HA provides and makes the interaction between people and their buildings easier. Thus, we may deal with very intuitive interfaces, which are usually embedded in any kind of object, allowing the environment to interact with different users and detect their presence.

In order to be able to apply the concepts of AmI and HA, applications must include different characteristics or technologies such as: (1) non-invasive hardware, (2) land/mobile communication infrastructures, (3) dynamic and distributed net devices, (4) human interfaces with very natural aspects and (5) managing all these technologies in a secure and reliable way [5]. Therefore, the combination of all these factors implies a serious technology challenge, especially in the development of new software to control and monitor all these systems by usable interfaces. However, current developments of these systems are carried out by programming them at a very low level of abstraction. Then, resulting systems may not be easily maintained and they are very tied to a specific technology, which quickly become obsolete.

Consequently, we must develop these systems from a more abstract viewpoint improving software quality and using more suitable technology in each case. Model-Driven Development (MDD) [10] is an emerging software engineering area that attempts to resolve these problems by introducing a software programming revolution. MDD focuses on the difference between a system design and its implementation so that the design may be carried out without taking into account implementation details, such as specific platforms or programming techniques. Then, using the abstract models, the code for the different platforms may be generated using model transformations. Therefore, the increasing complexity of different implementation platforms (such as .NET, Java/Jndi, OSGI[4], etc.) or programming domains (in our case, smart home control applications for KNX/EIB [2] or X10 installations) may be solved by using a higher degree of abstraction of the problem.

Thus, we propose the application of MDD to the development of Home Automation and Ambient Intelligence Sys-

*This project has been financed by CICYT, project number TIN2008-02985

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

IIES at EuroSys 2009, Sponsored by ACM SIGOPS. Nuremberg, Germany
Copyright 2009 ACM 978-1-60558-464-5 ...\$5.00.

tems (HAAIS). We present a software development process which allows the modelling of such systems. By means of proper mechanisms, we transform these models into Component Based Software and embedded devices implementations, paying special attention to the control, comfort and energy saving aspects. Moreover, the developed systems may be controlled by means of typical devices such as laptops, PDAs or mobile phones. This is possible by generating specific Web applications (front-end) for each device and AmI system (back-end).

The paper is organized as follow: Section 2 covers the main problems that appear using a traditional development process for HA and AmI systems. Section 3 presents our proposal to develop these kinds of systems using a MDD approach. On one hand, a specific DSL to model HA and AmI Systems (HAAIS-DSL) is presented. On the other hand, the model to code transformation carried out is described. Finally, Sections 4 and 5 present the related works and conclusions of the work respectively.

2. MOTIVATION

HAAI is a discipline that researches how to create an intelligent home environment. Nowadays, HAAI Systems are rapidly increasing. These systems are becoming more and more complex and the number of devices incorporated is rapidly increasing. Some of the tasks provided by HAAIS are:

- Setting the appropriate lighting in a room for a comfortable atmosphere and to save energy.
- Controlling the temperature in a specific room. It increases comfort but also implies to save energy.
- Finally, one of the most interesting characteristics of these kinds of systems is the remote control, so that that they may be controlled on-line with a mobile phone, PDA or laptop.

The aforementioned services are usually based on specific embedded hardware and software infrastructures (like KNX/EIB, X10, etc.). We can distinguish two kind of software to control a specific installation:

- physical installation control and configuration (back-end), coded at the lowest level and embedded.
- user front-end tools (web tools accessed from mobile phones, PDA or laptops).

For example, ETS (Engineering Tool Software) is a tool to configure intelligent home and building control installations made with the KNX/EIB infrastructure [2]. This tool is used at the lowest level of configuration (the physical codification) and it is specific for each system.

A HAAI system is composed of different devices so that complexity of the system is proportional to the number of devices incorporated. There are different kinds of devices: sensors, actuators, etc. These devices are interrelated, e.g. a light can be related with a push button, a timer or both of them. Again, complexity increases with the number of relations between different devices. Based on the relations among these devices, HAAI systems offer many possibilities to make life easier. In order to explain this better, we present an example in Figure 1.

In a public service building, saving energy is really important. Experience in these kinds of installations shows that once the lights are switched on, it is not usual to switch them off so that they are unnecessarily left on too much time. The automation of the lighting system assures that lights could be switched off when not needed. This method not only implies saving energy but it also makes the light bulbs last longer. The lights in the building could be automatically switched on in the morning, and switched off when the external light is bright enough. We may even consider that the building has different modules that will not necessarily have the same characteristics. On one hand, some modules are occupied earlier than others, so the lights in the different modules are not switched on at the same time. To obtain the desired scenarios in modules with different timetables, we will need a timer output for each module (in this example only one timer with two outputs is necessary). On the other hand, external light may be different in different rooms; it depends on the number of windows and the orientation, so it is necessary to have several light sensors. The lights have to be also divided into groups; light bulbs near the window will be usually switched off earlier than the rest and switched on later.

Thus, the following devices are needed in the example: timers, light-sensors and lights. The devices and the relations between them could be defined in KNX. Each device is identified by a physical address and group addresses are used to specify the relations.

The system presented is quite simple but then imagine that we also want to control the temperature in the building, there are more devices implied (heating, air conditioning, temperature sensor, blind...) and the relations become much more complex.

To implement the system back-end, the first issue we should address is to know how the system has to act in each circumstance. Once the functionality of the system is defined, it is important to identify the relations between the devices and how the devices should be integrated into the system. Both configuration and relations among devices (components of the system) must be hand coded at the lowest level, using physical address, group address, input signals, output signals, etc. This low level task is tedious and error prone. Then, when the back-end is configured the front-end to manage this specific system must be developed. The front-end depends on the physical installation and must be developed for each one. Then, adaptability becomes a key characteristic for these systems since installations are usually designed for the building needs.

A Model Driven Development approach may help to develop these kinds of systems facilitating their adaptability.

3. MODEL DRIVEN DEVELOPMENT APPROACH AND HAAIS-DSL

This section presents how MDD (Model Driven Development) and DSL (Domain Specific languages) engineering may solve practical problems presented by Home Automation and Ambient Intelligence Systems. Figure 2 summarizes our proposal to develop HAAIS based on a Model Driven Development. Firstly, we model the system using a DSL called HAAIS-DSL (figure 2-1). The models must provide a clear picture of the system defined at a high level of abstraction. Secondly, we define a model to code transformation to

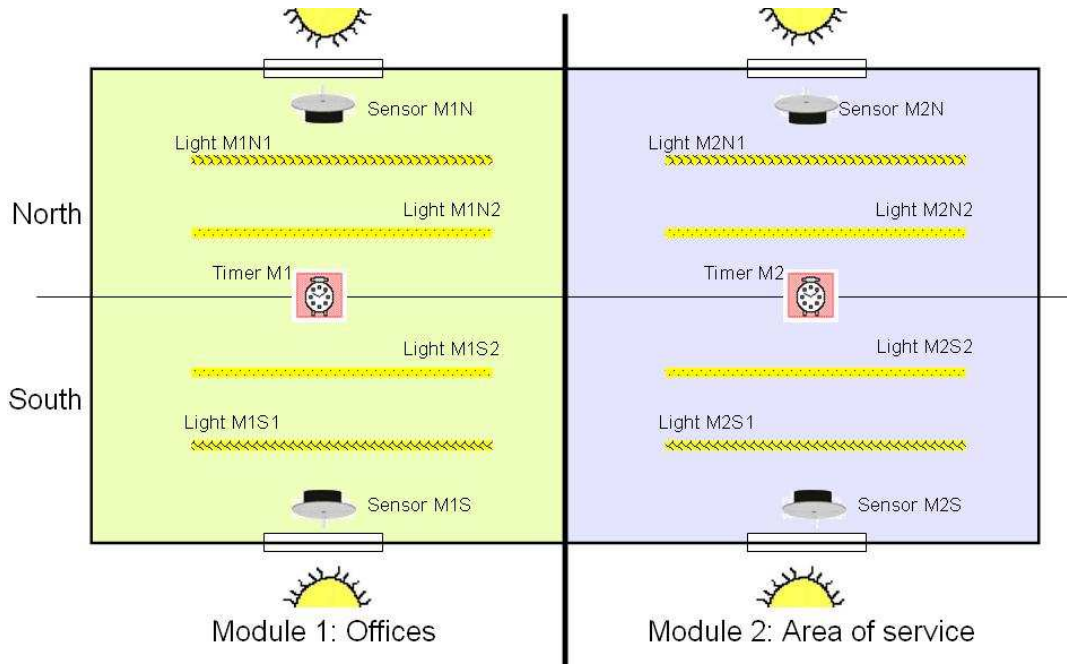


Figure 1: Location schema of several devices in a public building

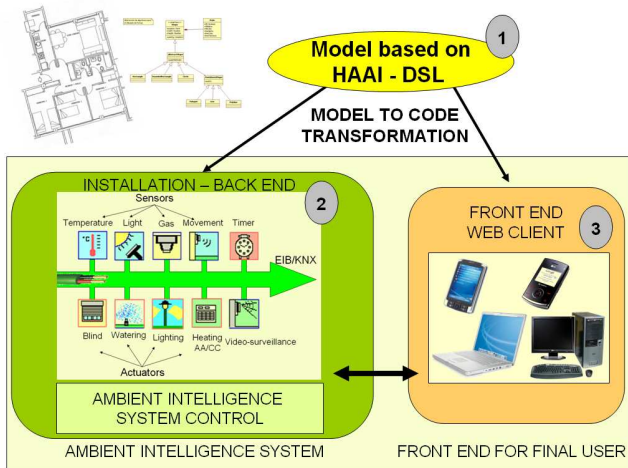


Figure 2: Model Driven Development approach overview

generate both the back-end system (figure 2-2) and the front-end system (figure 2-3) using as input the model designed.

HAAIS-DSL allows us to graphically represent the system (describing its model) and to automatically generate the system application by using model transformations. Then, different applications may be easily created for different installations just defining (graphically) a new model for such installations, highly improving system adaptability.

This technology reduces the dependency of the software development process on the final execution platforms. In following subsections we describe each phase of our development process, beginning with the HAAIS-DSL definition.

3.1 HAAIS-DSL

MDE y DSL share the idea that language engineering may help in domain modelling. DSL Engineering is positioned at a higher abstract level, using different technical solutions like MDE, Grammarware, XML solutions, etc. MDE mainly uses metamodeling capabilities to implement families of languages in specific application *domains* [8].

On one hand, MDE and DSLs are based on the concept of *technical space* [12]. A *technical space* is a model management framework with a set of tools that operate on the models definable within the framework. It represents specific working context with specific implementation technologies, tools and approaches, where applications are specified, instantiated and utilized from various tools and engines [12]. On the other hand, *domains* represent contexts via specific application aspects and not by given programming language concepts. Thus, *technical spaces* and *domains* are comparable when creating contexts as metamodels. While the former is focused on the technological and implementation issues, the latter focuses on conceptual application representations.

There are strong relations between DSLs and models [8]. We have used a typical *technical space* definition based on MetaMetaModel (M3), MetaModel (M2) and Model (M1) to define our *technical space* which is designed on Ecore (M3)[6], HAAIS-DSL MetaModel (M2) and HAAIS-Model (M1) as can be observed in figure 3-left.

The HAAIS-DSL (see figure 3-right) conforms to Ecore and it allows us to model the system components, their functionalities and their relations with others components. To understand this DSL we need to know the different kinds of devices:

- Light sensors, temperature sensors, movement sensor, timer, buttons, etc. These devices have one or more *outputs* that allow the activation of different *inputs* to carry out an action in the system.

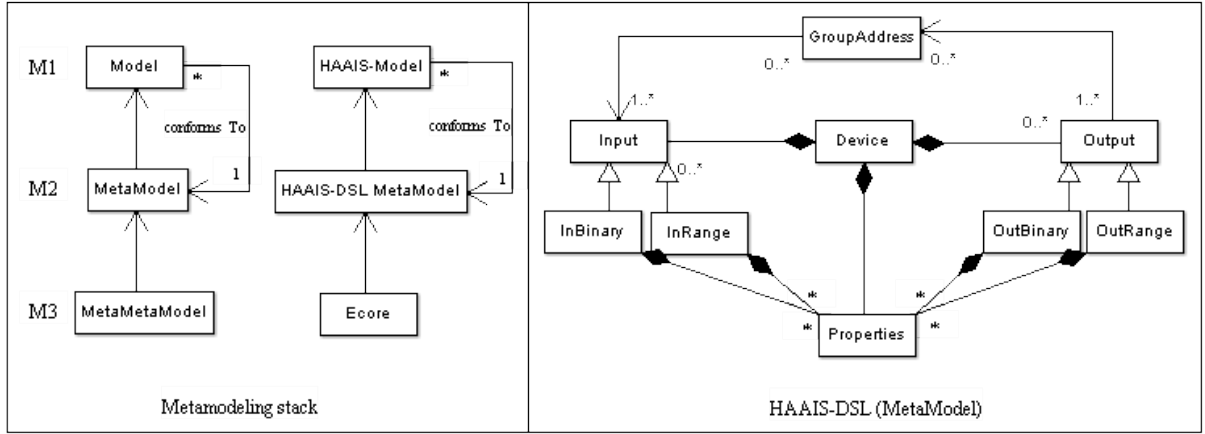


Figure 3: MetaModeling Stack (left) and HAAIS-DSL Metamodel (right)

- Actuators like lights, blinds, air conditioning, heating, etc. are devices with one or more *inputs*. These devices carry out an action when an *output* is activated in the system.
- Devices can exist with both *inputs* and *outputs*. For example, a camera with a movement sensor may act as a sensor, when it detects movement (*output*), or as an actuator, recording (*input*) when a signal is received.

Each kind of device has some specific *properties*, but all of them have a *physical address* that identifies the device. The devices on the system are related among them using a “*group address*”, which indicates the *inputs* and *outputs* of the inter-related devices. A *group address* may contain more than two *inputs/outputs* and each *input/output* can be introduced into more than one *group address*.

For example, to model a light-sensor device we will need to have some *properties* like the *physical address* and the *state*. The *output* of one light-sensor has several properties like *captured_value* which represents the exterior lighting and *sensitivity* which indicates the level when the desired brightness is reached. In next section, we model the previous example (presented in Section 2). In particular, the light-sensor called *SensorMIN* is also modeled following this description in figure 1.

3.1.1 Model a system using HAAIS-DSL

In section 2 an example of building automation was presented (figure 1). According to the DSL defined in figure-3-right we can model the system obtaining a model that can be observed in figure 4.

The building we want to control is divided into two different areas: an area is dedicated to offices and the other one is a service area to attend to the public. The timetable of these areas is not the same, for this reason the building is divided into two modules.

In order to implement the system, an *output* of a timer is needed to control the time which the lights have to be switched on. Then we need just one *timer* with two *outputs*. In this example, lighting in the building has to be bright enough during the working day, so that if the external light is not enough the sensor will generate an

output signal to switch on the lights. Several sensors are also necessary since the number of windows and the orientation of the place have an effect on the external light. Lights can be divided into several groups, because of the different light intensity between different zones (it is usually lighter near the windows).

To model this example, firstly we identify the *devices*, the *inputs* and *outputs* that take place in the system and the relation established between them:

- *Inputs*: each light has an input.
- *Outputs*: an output of each light sensor and two outputs of a timer are needed.

Moreover, a *group address* is necessary for each set of devices that share the same *output*, i.e. they have the same behaviour.

Figure 4 represents the model for this system that conforms to the DSL defined. Once the model is represented, the next step is to get the specific application for this system (front-end and back-end) on specific embedded operating environment, applying code generation techniques and model transformations.

3.2 Generating the system code and their configurations: Model to code transformations

We produce the whole application (back-end and front-end) from the graphical design applying model transformations to automatically generate the code and to avoid hand code. By applying these transformations to the model based on HAAIS-DSL, we obtain a HA system which includes: back-end characteristics, such as database SQL specification or code to control the EIB/KNX devices; and front-end functionalities, mainly a Web application to control the system installed. In particular, the model transformations (figure 2) were carried out using ATLAS Transformation Language (ATL) [7] and MofScript[3]. Fundamentally, the model transformation is based on transformation rules. These transformation rules allow us to navigate over the model structure which is based on Ecore. An example of model transformation may be observed in figure 5 where *devices* and *groupAddress* are processed.

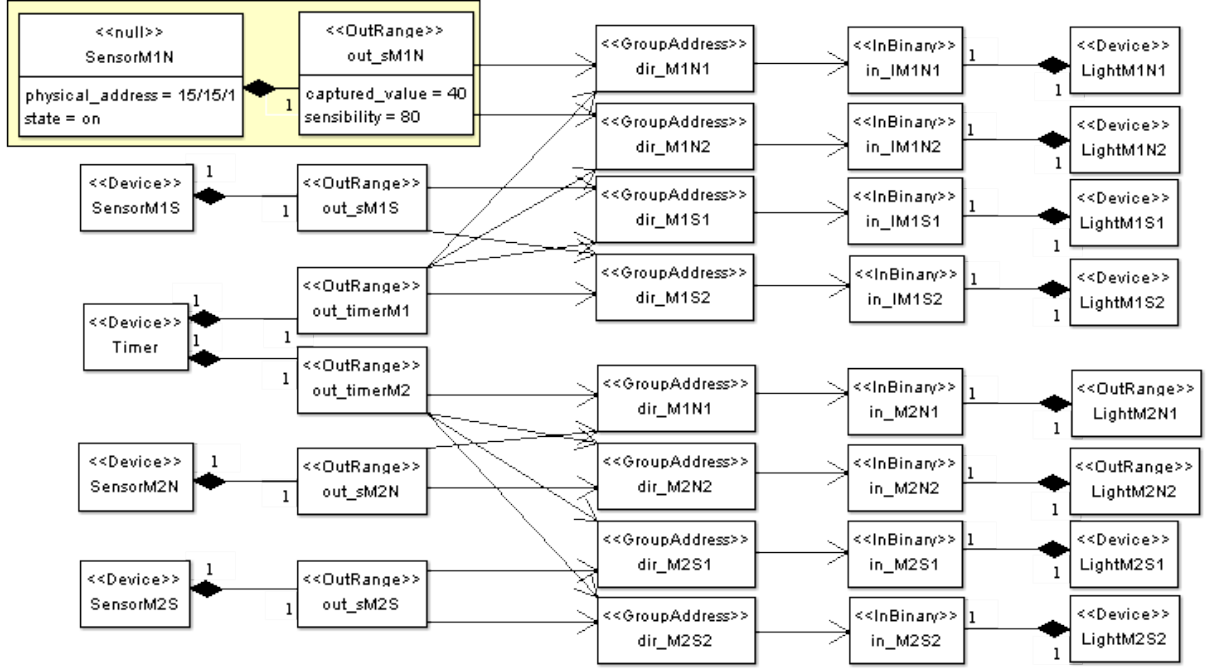


Figure 4: Model of the system previously described at section 2

```

texttransformation Ecore2XMLconfigurations
(in model_in:'haais_mm') { // HAAIS metamodel
  model_in.EPackage::main() {
    file('system_configuration.xml')
    model_in.objectsOfType(model_in.EClass) -> forEach
(ecc1) {
      ecc1.devices() // procesing devices information
    }
    model_in.objectsOfType(model_in.EReference)-> forEach
(ref) {
      ref.groupAddress() // processing group address
    }
    self.createConfiguration()
    ...
  }
}

```

Figure 5: Example of model transformation using Mofscript

The architecture software, the code and configuration automatically generated for the embedded devices can be observed on figure 6 and includes the following¹:

- SQL Database schema, configuration and data.
- Complete configuration devices and group devices which are managed by DomoServer (an specific software layer to manage EIB devices).
- Web site implemented on PHP and Ajax to manage the devices installed on the system. This Web site is accessible by laptops, PDAs, mobile phones, etc.

4. RELATED WORKS

¹We do not include the code generated due to lack of space in the paper.

Currently, several multinational companies like Siemens, Phillips, etc. are investing money in large quantities to develop these kinds of systems (HAAIS) and they have proposed several technological approaches at the lowest level: KNX/EIB[2], X10, OSGI [4] (for example, systems OSGI written in Java), etc.

In [9], the authors suggest the application of an MDE approach to design their systems (Wireless Sensor Network Applications) at different abstraction levels, providing automatic model transformations to incrementally refine abstract models into more concrete ones. They show us how a model can be managed using several tools like GMF, ATL, etc, which we are using to develop our approach.

In AMPLE Project [1] is developing a Smart Homes case study using several software engineering technologies like aspect-orientation, model-driven development or software product lines. However, currently there are not a complete prototype to generate these kind of systems.

5. CONCLUSIONS AND FUTURE WORK

This paper presents a Model Driven Development process to develop Home Automation and Ambient Intelligence Systems. Thus, HAAIS-DSL tackle the complexity of these kind of systems, facilitating the specific domain elements and the rules required to model these systems. Models guide the development process making possible the description and configuration of complex systems.

The models obtained can be transformed using model to code transformations to obtain both, the back-end system configuration (C# code) and the front-end tools for the final users. The front-end tools are based on Web Technologies (PHP and Ajax) allowing the use of several kind of client devices like PDA, mobile phones or laptops. The code and configuration generation decreases the hand written code

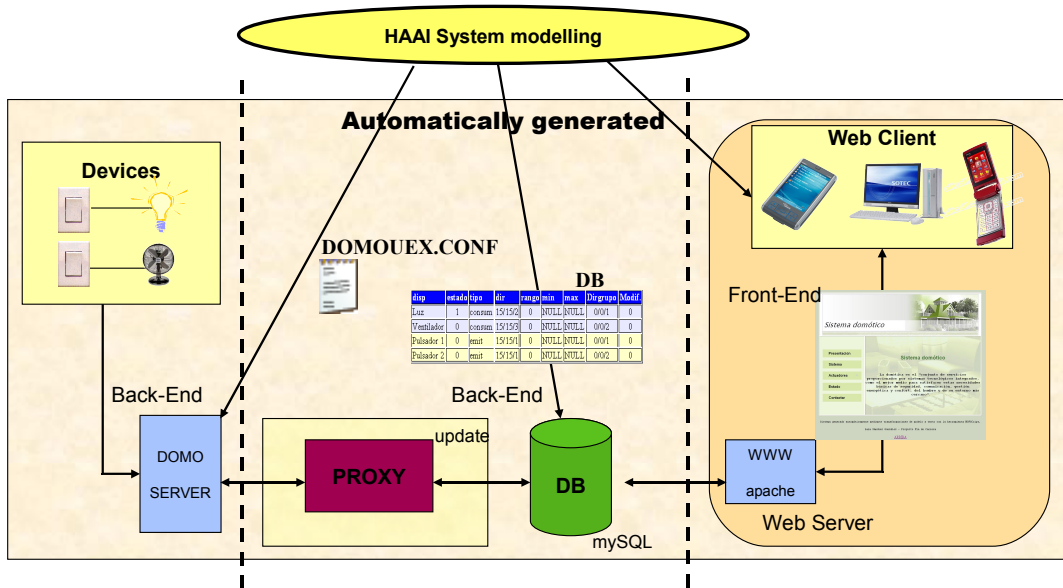


Figure 6: HAAI software architecture and code generated to manage embedded devices

and the number of errors. Additionally, the code could be optimized and generated using better practices like designs pattern.

Model Driven Development can be used in other kind of embedded software systems. So, using a MDD process the complexity of specific technology can be reduced since the system could be automatically generated from models. Moreover, the complexity of the integration of embedded devices on a system and their configuration can be managed.

Currently, we are working on several strategic lines like extending HAAIS-DSL with more kinds of input and output signals to obtain a more flexible DSL.

6. REFERENCES

- [1] AMPLÉ Project (Aspect Oriented, Model Driven and Product Line Engineering). <http://ample.holos.pt>, December 2008.
- [2] KNX/EIB Technologies. <http://www.knx.org/>, December 2008.
- [3] MOFScript Project. <http://www.eclipse.org/gmt/mofscript/>, 2008.
- [4] T. O. Alliance. OSGi Service Platform — Core Specification, Aug. 2005. Release 4.
- [5] K. Ducatel, M. Bogdanowicz, F. Scapolo, J. Leijten, and J. Burgelma. STAG: Scenarios for ambient intelligence in 2010. Technical report, ISTAG 2001 Final Report. <ftp://ftp.cordis.lu/pub/ist/docs/istagscenarios2010.pdf>, 2001.
- [6] EMF. EMF. Eclipse Modeling Home. <http://www.eclipse.org/modeling/emf/>, 2008.
- [7] F. Jouault and I. Kurtev. Transforming Models with ATL. In J.-M. Bruel, editor, *MoDELS Satellite Events*, volume 3844 of *Lecture Notes in Computer Science*, pages 128–138. Springer, 2005.
- [8] I. Kurtev, J. Bézin, F. Jouault, and P. Valduriez. Model-based DSL frameworks. In P. L. Tarr and W. R. Cook, editors, *OOPSLA Companion*, pages 602–616. ACM, 2006.
- [9] F. Losilla, C. Vicente-Chicote, B. Álvarez, A. Iborra, and P. Sánchez. Wireless Sensor Network Application Development: An Architecture-Centric MDE Approach. In F. Oquendo, editor, *ECSA*, volume 4758 of *Lecture Notes in Computer Science*, pages 179–194. Springer, 2007.
- [10] B. Selic. The Pragmatics of Model-Driven Development. *IEEE Software*, 20(5):19–25, 2003.
- [11] N. Shadbolt. Ambient Intelligence. *IEEE Intelligent Systems*, 18(4):2–3, 2003.
- [12] A. Staikopoulos and B. Bordbar. A Metamodel Refinement Approach for Bridging Technological Spaces, a Case Study. 4th Workshop in Software Model Engineering (WiSME 2005) - in conjunction with MoDELS/UML, 2005.